1882. Process Tasks Using Servers

Daily Question - November 7, 2026

Problem Statement

You are given two 0-indexed integer arrays servers and tasks of lengths n and m respectively. servers[i] is the weight of the ith server, and tasks[j] is the time needed to process the jth task in seconds.

Tasks are assigned using a task queue. Initially, all servers are free and the queue is empty.

At second j, the jth task is inserted into the queue, starting with task 0 at second 0. As long as free servers and queued tasks exist, assign the front task to a free server with the smallest weight; break weight ties by the smallest server index.

If no server is free, wait until one becomes free and immediately assign the next task. If multiple servers become free simultaneously, assign queued tasks in insertion order using the same weight and index priority.

A server assigned task j at second t becomes free again at second t + tasks[j]. Return ans, where ans[j] is the index of the server assigned to task j.

Examples

Example 1: servers = [3,3,2], tasks = [1,2,3,2,1,2]. Output: [2,2,0,2,1,2].

Example 2: servers = [5,1,4,3,2], tasks = [2,1,2,4,5,2,1]. Output: [1,4,1,4,1,3,2].

Constraints

servers.length == n; tasks.length == m; 1 <= n, m <= 200000; 1 <= servers[i], tasks[j] <= 200000.

Solution: Two Min-Heaps

Maintain two priority queues. The free-server heap stores [weight, index] and orders by smallest weight, then smallest index. The busy-server heap stores [freeTime, weight, index] and orders by earliest free time, then smallest weight, then smallest index.

For each task j, set currentTime = max(currentTime, j). First move every busy server whose freeTime is at most currentTime back to the free heap. If no server is free, advance currentTime to the earliest busy-server freeTime and move all servers freeing at that exact time back to the free heap.

Then remove the best free server, record its index for the current task, and insert it into the busy heap with freeTime = currentTime + tasks[j].

Complete Java Solution

```
import java.util.*;

class Solution {
    public int[] assignTasks(int[] servers, int[] tasks) {
        int taskCount = tasks.length;
```

```
        int[] answer = new int[taskCount];

        PriorityQueue<int[]> freeServers = new PriorityQueue<>(
                (first, second) -> {
                    if (first[0] != second[0]) {
                        return Integer.compare(first[0], second[0]);
                    }
                    return Integer.compare(first[1], second[1]);
                }
        );

        PriorityQueue<int[]> busyServers = new PriorityQueue<>(
                (first, second) -> {
                    if (first[0] != second[0]) {
                        return Long.compare(first[0], second[0]);
                    }
                    if (first[1] != second[1]) {
                        return Integer.compare(first[1], second[1]);
                    }
                    return Integer.compare(first[2], second[2]);
                }
        );

        for (int index = 0; index < servers.length; index++) {
            freeServers.offer(new int[] {servers[index], index});
        }

        long currentTime = 0;

        for (int taskIndex = 0; taskIndex < taskCount; taskIndex++) {
            currentTime = Math.max(currentTime, taskIndex);

            while (!busyServers.isEmpty()
                    && busyServers.peek()[0] <= currentTime) {
                int[] server = busyServers.poll();
                freeServers.offer(
                        new int[] {server[1], server[2]}
                );
            }

            if (freeServers.isEmpty()) {
                currentTime = busyServers.peek()[0];

                while (!busyServers.isEmpty()
                        && busyServers.peek()[0] == currentTime) {
                    int[] server = busyServers.poll();
                    freeServers.offer(
                            new int[] {server[1], server[2]}
                    );
                }
            }

            int[] server = freeServers.poll();
            answer[taskIndex] = server[1];

            busyServers.offer(new int[] {
                    (int) (currentTime + tasks[taskIndex]),
                    server[0],
                    server[1]
            });
        }

        return answer;
    }
}
```

Correctness

The free heap implements the required selection rule: among available servers, it always selects the minimum weight and, for equal weights, the minimum index. The busy heap orders servers by release time, then by the same server priority, so when several servers become free simultaneously they are restored to the free heap in the required order.

Tasks arrive in increasing time order. For task j, first release all servers whose free time is no later than the current time. If none are available, advancing to the earliest release time correctly models waiting for the next available server. The selected server processes the task until currentTime + tasks[j], after which it becomes available again.

Complexity

Time: O(n + m log n). Building the free heap takes O(n), and each task requires O(log n) heap operations. Auxiliary space: O(n), for the two heaps and the result array.

Implementation Notes

Use long for currentTime because the total completion time can exceed the int range: with m and task duration both up to 200000, the final free time can be as large as 4 * 10^10. The busy heap stores the free time as int only because the problem constraints permit it in this implementation; a long-based heap entry would be safer for general inputs. The final answer is the assigned server index for each task.